

Lista de Atividades

Introdução ao ROOT: TTree, TFile e Ajuste de Histogramas

Marco Dias

Instruções gerais de entrega

- A atividade deve ser resolvida usando o framework ROOT (macros em C++/CINT ou ROOT-CLING, com extensão `.C`, ou um pequeno executável compilado ligado à biblioteca do ROOT).
- Crie uma pasta `ex_root/` no seu repositório, contendo os dois scripts pedidos, o arquivo `.root` gerado (ou instruções claras de como regenerá-lo) e a figura final exportada (por exemplo em `.png` ou `.pdf`).
- Inclua um `README.md` explicando como rodar cada script, por exemplo:

```
root -l generate.C
root -l plot.C
```

- Faça o `commit` e `push` de todos os arquivos (scripts, figura e, se possível, o arquivo `.root`) para o seu repositório público antes do prazo de entrega.

1 ROOT, TTree e TFile

1.1 Contexto

O ROOT é o framework de análise de dados utilizado pela grande maioria dos experimentos de física de altas energias para armazenar, processar e visualizar grandes volumes de dados. Dois de seus objetos mais fundamentais são:

- **TFile**: representa um arquivo em disco (formato binário próprio do ROOT) capaz de armazenar de forma eficiente e comprimida qualquer objeto do ROOT – histogramas, árvores, gráficos, etc. É análogo a um arquivo de dados de um experimento real, onde eventos reconstruídos são gravados para análise posterior.
- **TTree**: representa uma estrutura de dados tabular, otimizada para armazenar um grande número de "eventos", cada um com um ou mais "ramos" (*branches*). Cada branch pode conter uma variável simples (um `double`, um `int`) ou estruturas mais complexas (vetores, objetos definidos pelo usuário, como vimos com a classe `MyEvent` em aula). O **TTree** é a estrutura padrão usada para armazenar dados de colisões evento a evento em experimentos como o CMS ou o ATLAS.

Já a geração de números aleatórios com uma distribuição de probabilidade específica (por exemplo, a distribuição normal/gaussiana) é uma ferramenta central em simulações de Monte Carlo, seja para gerar pseudo-dados de teste, seja para modelar processos físicos e efeitos de resolução experimental (como a resposta de um calorímetro a uma energia depositada). O ROOT fornece a classe **TRandom3** (baseada no gerador Mersenne Twister) com o método `Gaus(media, sigma)` para esse fim.

1.2 Exercício 1 – Geração de dados e gravação em TTree/TFile

Escreva uma macro ROOT, `generate.C`, que:

- Crie um objeto `TFile` chamado `dados.root`, aberto em modo de escrita/recriação ("RECREATE").
- Crie um objeto `TTree` chamado, por exemplo, `tree`, com um branch de nome `x` do tipo `double`.
- Usando um objeto `TRandom3`, gere $N = 1000$ números aleatórios distribuídos normalmente (gaussianos), com média e desvio-padrão de sua escolha (por exemplo $\mu = 0$ e $\sigma = 1$), preenchendo o branch `x` do `TTree` a cada número gerado (`tree->Fill()`).
- Escreva a árvore no arquivo (`tree->Write()`) e feche o `TFile` corretamente ao final da macro.

Esqueleto sugerido:

Listing 1: `generate.C` – esqueleto

```
1 void generate() {
2     const int N = 1000;
3     double mean = 0.0;
4     double sigma = 1.0;
5
6     TFile *file = new TFile("dados.root", "RECREATE");
7     TTree *tree = new TTree("tree", "Numeros aleatorios gaussianos");
8
9     double x;
10    tree->Branch("x", &x, "x/D");
11
12    TRandom3 rnd(0); // semente 0 = baseada no relógio
13    for (int i = 0; i < N; i++) {
14        x = rnd.Gaus(mean, sigma);
15        tree->Fill();
16    }
17
18    file->cd();
19    tree->Write();
20    file->Close();
21 }
```

Requisitos de entrega: o arquivo `generate.C` deve ser executável diretamente com `root -l generate.C` e produzir o arquivo `dados.root` contendo o `TTree` com os $N = 1000$ valores.

1.3 Exercício 2 – Leitura, histograma e ajuste

Escreva uma segunda macro ROOT, `plot.C`, que:

- Abra o arquivo `dados.root` produzido no Exercício 1 (modo "READ") e recupere o `TTree` salvo.
- Percorra a árvore evento a evento (usando `tree->GetEntry(i)`, análogo ao que vimos com `getline` para arquivos texto) e preencha um histograma `TH1F` com os valores do branch `x`. Alternativamente, você pode preencher o histograma diretamente com `tree->Draw("x >> h")`, mas nesse caso explique no `README.md` por que esse atalho funciona.
- Configure a aparência do histograma:

- linha de contorno **preta, sólida** e mais **grossa** que o padrão – use `SetLineColor(kBlack)`, `SetLineStyle(1)` e `SetLineWidth(...)` com uma espessura maior que 1 (por exemplo 2 ou 3);
 - área do histograma preenchida com a cor **amarela** – use `SetFillColor(kYellow)`;
 - fundo (*background*) do canvas em cor **branca** – configure isso no objeto `TCanvas` (por exemplo `SetFillColor(kWhite)`) e/ou com `gStyle->SetCanvasColor(kWhite)`;
 - rótulos (*labels*) para os eixos x e y (por exemplo "Valor gerado" e "Numero de entradas"), usando `SetXTitle/SetYTitle` ou o terceiro argumento do construtor do `TH1F`.
- (d) Ajuste (*fit*) o histograma a uma função gaussiana, usando `TH1::Fit("gaus")`, e verifique que os parâmetros obtidos pelo ajuste (média e desvio-padrão) são compatíveis com os valores usados para gerar os dados no Exercício 1.
- (e) Desenhe o resultado em um `TCanvas` e salve a figura final em um arquivo de imagem (por exemplo `histograma.png` ou `histograma.pdf`), usando `TCanvas::SaveAs(...)`.

Esqueleto sugerido:

Listing 2: plot.C – esqueleto

```

1 void plot() {
2     TFile *file = new TFile("dados.root", "READ");
3     TTree *tree = (TTree*) file->Get("tree");
4
5     double x;
6     tree->SetBranchAddress("x", &x);
7
8     TH1F *h = new TH1F("h", "Distribuicao gerada;Valor gerado;Numero de
9         entradas",
10         50, -5, 5);
11
12     Long64_t nentries = tree->GetEntries();
13     for (Long64_t i = 0; i < nentries; i++) {
14         tree->GetEntry(i);
15         h->Fill(x);
16     }
17
18     h->SetLineColor(kBlack);
19     h->SetLineStyle(1);
20     h->SetLineWidth(3);
21     h->SetFillColor(kYellow);
22
23     TCanvas *c = new TCanvas("c", "Histograma", 800, 600);
24     c->SetFillColor(kWhite);
25
26     h->Fit("gaus");
27     h->Draw();
28
29     c->SaveAs("histograma.png");
30
31     file->Close();
32 }
```

Requisitos de entrega:

- o arquivo `plot.C` deve ser executável diretamente com `root -l plot.C` (supondo que `dados.root` já exista);

- a figura final (`histograma.png` ou similar) deve ser incluída no repositório;
- no `README.md`, reporte os valores de média e desvio-padrão obtidos pelo ajuste gaussiano (impressos no terminal pelo próprio `Fit`, ou lidos com `TF1::GetParameter(...)`) e compare-os com os valores usados na geração dos dados no Exercício 1.

Sugestão de estrutura de diretórios

```
f056-atividade-root/  
+-- generate.C  
+-- plot.C  
+-- dados.root          (gerado pelo generate.C)  
+-- histograma.png      (gerado pelo plot.C)  
+-- README.md
```